

SE4-DMAD — Kursusnoter

Diskret Matematik, Algoritmer og Datastrukturer

Casper Holm Bach

Indhold

1	Logik	4
1.1	Logiske operatorer og præcedens	4
1.2	Sandhedstabeller	4
1.3	Logiske ækvivalenser	4
1.4	Kvantorer og indlejrede kvantorer	4
1.5	Eksamenstips og faldgruber	4
2	Bevismetoder	4
2.1	Direkte bevis	4
2.2	Kontrapositionsbevis	4
2.3	Modstridsbevis	4
2.4	Induktionsbevis	4
2.5	Eksamenstips og faldgruber	4
3	Relationer	4
3.1	Egenskaber	4
3.2	Ækvivalensrelationer	4
3.3	Partielle ordninger	4
3.4	Lukninger	4
3.5	Eksamenstips og faldgruber	4
4	Asymptotisk analyse	4
4.1	Hvad er en algoritme?	4
4.2	Analyse af en algoritme	5
4.3	Best, average og worst case	5
4.4	Definitioner	5
4.5	Sammenligning af voksehastigheder	8
4.6	Vækstrategierarki	9
4.7	Invarianter	10
4.8	Analyse af løkker i hånden	11
4.9	Eksamenstips og faldgruber	11
5	Rekursionsligninger	11
5.1	Substitutionsmetoden	11
5.2	Rekursionstræ-metoden	11
5.3	Master-sætningen (CLRS 4. udgave)	11
5.4	Eksamenstips og faldgruber	11
6	Grafgennemløb	11
6.1	Repræsentationer	11
6.2	Bredde-først søgning (BFS)	11
6.3	Dybde-først søgning (DFS)	11
6.4	Topologisk sortering	11
6.5	Eksamenstips og faldgruber	11
7	Sortering	11
7.1	Sammenligningsbaserede sorteringer	11
7.2	Lineærtids-sorteringer	11
7.3	Sammenligningstabel	11
7.4	Eksamenstips og faldgruber	11
8	Minimum udspændende træ	12
8.1	Snit-egenskaben (cut property)	12
8.2	Kruskals algoritme	12
8.3	Prims algoritme	12
8.4	Eksamenstips og faldgruber	12
9	Disjunkte mængder (Union-Find)	12
9.1	Operationer	12

9.2	Union by rank	12
9.3	Path compression (stikomprimering)	12
9.4	Eksamenstips og faldgruber	12
10	Grådige algoritmer og Huffman-kodning	12
10.1	Det grådige princip	12
10.2	Huffman-kodning	12
10.3	Eksamenstips og faldgruber	12
11	Søgetræer	12
11.1	Binære søgetræer	12
11.2	Rød-sorter træer	12
11.3	Indsættelse og rotationer	12
11.4	Udvidede (augmented) træer	12
11.5	Eksamenstips og faldgruber	12
12	Heaps og prioritetskøer	13
12.1	Heap-egenskaben og array-repræsentation	13
12.2	Opbygning af en heap	13
12.3	Operationer	13
12.4	Eksamenstips og faldgruber	13
13	Dynamisk programmering	13
13.1	Principper	13
13.2	Gennemregnede eksempler	13
13.3	Køretids- og pladsanalyse	13
13.4	Eksamenstips og faldgruber	13
14	Hashing og ordbøger (dictionaries)	13
14.1	Kædning (chaining)	13
14.2	Åben adressering	13
14.3	Fyldningsgrad (load factor) og ydeevne	13
14.4	Eksamenstips og faldgruber	13
15	Korteste veje	13
15.1	Relaksering (relaxation)	13
15.2	Dijkstras algoritme	13
15.3	Bellman-Ford	13
15.4	Korteste veje i en DAG	13
15.5	Hvilken algoritme skal bruges	13
15.6	Eksamenstips og faldgruber	14

1 Logik

1.1 Logiske operatorer og præcedens

Noter tilføjes ($\neg \wedge \vee \Rightarrow \Leftrightarrow \oplus$, præcedenshierarki).

1.2 Sandhedstabeller

Noter tilføjes ($\#rækker = 2^{\text{antal variable}}$).

1.3 Logiske ækvivalenser

Noter tilføjes (f.eks. $p \Rightarrow q \equiv \neg p \vee q$, De Morgan; tautologi / modstrid / kontingens).

1.4 Kvantorer og indlejrede kvantorer

Noter tilføjes ($\forall, \exists$; rækkefølgen betyder noget; negation af kvantificerede udsagn).

1.5 Eksamenstips og faldgruber

Noter tilføjes.

2 Bevismetoder

2.1 Direkte bevis

Noter tilføjes.

2.2 Kontrapositionsbevis

Noter tilføjes.

2.3 Modstridsbevis

Noter tilføjes.

2.4 Induktionsbevis

Noter tilføjes (basis + induktionsskridt; genkende et gyldigt vs. ugyldigt bevis).

2.5 Eksamenstips og faldgruber

Noter tilføjes.

3 Relationer

3.1 Egenskaber

Noter tilføjes (refleksiv, symmetrisk, antisymmetrisk, transitiv).

3.2 Ækvivalensrelationer

Noter tilføjes (refleksiv + symmetrisk + transitiv; ækvivalensklasser).

3.3 Partielle ordninger

Noter tilføjes (refleksiv + antisymmetrisk + transitiv).

3.4 Lukninger

Noter tilføjes (refleksiv / symmetrisk / transitiv lukning).

3.5 Eksamenstips og faldgruber

Noter tilføjes.

4 Asymptotisk analyse

4.1 Hvad er en algoritme?

En algoritme er en betegnelse for en trinvis opskrift for hvordan en computer kan løse en given opgave eller problem. Den tager imod et input, bearbejder det, og returnerer et output, som er resultatet af algoritmen.

Mindstekravet for algoritmer er at de kan betegnes som **korrekte**:

- Algoritmen er ikke en uendelig løkke, og den stopper for inputs.
- Algoritmen giver et korrekt svar på det problem vi forsøger at løse med den.

En korrekt algoritme kan derudover have forskellige kvaliteter:

- Hastighed
- Pladsforbrug (RAM / Memory)
- Komplexitet af implementation
- Ekstra problemspecifikke egenskaber

4.2 Analyse af en algoritme

Vi er ofte interesseret i at måle disse kvaliteter af en algoritme. For at kunne analysere en algoritme skal man bruge følgende:

- Model af problem
 - Definer input og output.
- Model af maskine
 - Hvilket “type” computer kører algoritmen på? I dette kursus bruges RAM-modellen, hvor grundlæggende operationer tager konstant tid.
- Mål for kvalitet
 - I dette kursus fokuserer vi på at analysere tidsforbrug.
- Matematiske værktøjer
 - Asymptotisk notation til at beskrive køretidsvækst.
 - Invarianter der skal være sande under hele kørslen.
 - Induktion som bevisteknik.
 - Rekursionsligninger der beskriver køretid for rekursive algoritmer.

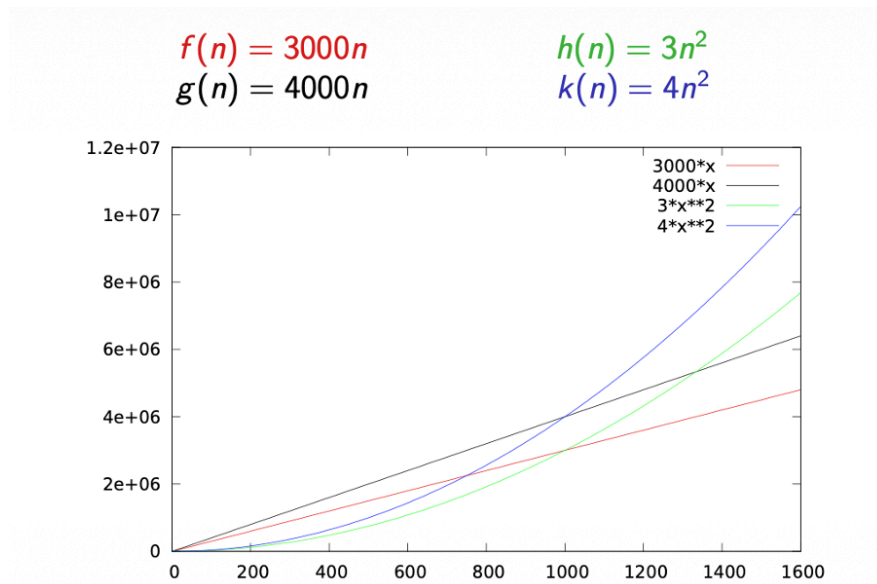
4.3 Best, average og worst case

Vi kan generelt måle en algoritmes hastighed i enten best, average eller worst case. Dog giver det bedst mening at måle i **worst case**-køretid, da dette giver os en garanti for hvad vi kan forvente. En worst case er ofte en voksende funktion.

4.4 Definitioner

Før vi begynder at fremstille en algoritme der kan løse vores problem, vil vi gerne kende voksehastigheden af de forskellige algoritmer så vi kan vælge den hurtigste. Det gør vi i det her kursus med RAM-modellen, som er bevist at afspejle rigtig køretid ret godt.

Når vi sammenligner forskellige voksehastigheder, er konstanter der bliver ganget på ligegyldige. Det vil altid være ledet der indebærer voksehastigheden (det led hvor n befinder sig), som dominerer.



Figur 1: På grafen ser vi at funktionerne $h(n)$ og $k(n)$ altid vil overhale $f(n)$ og $g(n)$, selvom deres konstant er betydelig mindre.

For at simplificere analysen af algoritmer bruger vi asymptotisk notation. Ved asymptotisk notation betragter vi alle konstanter der bliver ganget på for ens (c).

$$\{c \cdot f(n) \mid \text{alle } c > 0\}$$

Figur 2: Det vil sige alle algoritmer med samme voksehastighed bliver betragtet som lige gode.

Vi kan nu bruge følgende relationer til at sammenligne voksehastigheder, som hver især har en betegnelse:

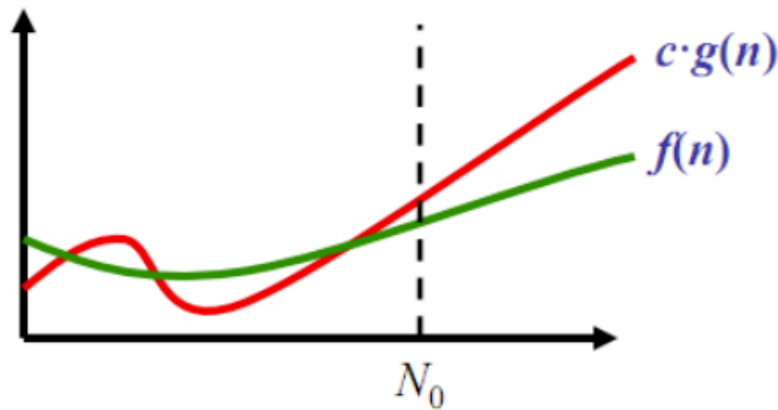
Relation	Betegnelse	Udtales
$\leq$	O	O
$\geq$	Ω	Omega
$=$	Θ	Theta
$<$	o	Lille o
$>$	ω	Lille omega

4.4.1 O (øvre grænse)

$$f(n) = O(g(n))$$

Hvis der findes en $c > 0$ og grænse N_0 , hvor der for alle n efterfølgende ($n \geq N_0$), gælder:

$$f(n) \leq c \cdot g(n)$$



Figur 3: $g(n)$ sætter loftet for hvor hurtigt $f(n)$ må vokse.

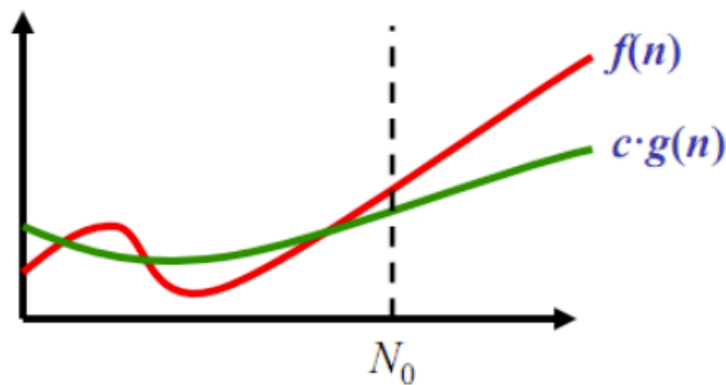
Det betyder altså at $g(n)$ sætter loftet for, hvor hurtigt $f(n)$ må vokse. $f(n)$ må gerne vokse samme hastighed, som $g(n)$. Der er ikke noget loft for hvor meget hurtigere $g(n)$ må vokse end $f(n)$.

4.4.2 Ω (nedre grænse)

$$f(n) = \Omega(g(n))$$

Hvis der findes en $c > 0$ og grænse N_0 , hvor der for alle n efterfølgende ($n \geq N_0$), gælder:

$$f(n) \geq c \cdot g(n)$$



Figur 4: $g(n)$ sætter gulvet for hvor hurtigt $f(n)$ skal vokse.

Det betyder altså at $g(n)$ sætter gulvet for, hvor hurtigt $f(n)$ skal vokse. $f(n)$ må gerne vokse samme hastighed, som $g(n)$. Der er ikke noget loft for hvor meget hurtigere $f(n)$ må vokse end $g(n)$.

4.4.3 Θ

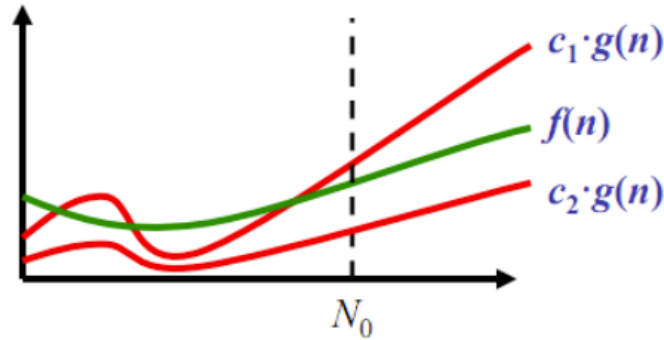
$$f(n) = \Theta(g(n))$$

Hvis der findes konstanter $c_1, c_2 > 0$ og en grænse N_0 , hvor der for alle n efterfølgende ($n \geq N_0$), gælder:

$$f(n) = O(g(n)) \quad \text{og} \quad f(n) = \Omega(g(n))$$

Som også kan beskrives:

$$c_2 \cdot g(n) \leq f(n) \leq c_1 \cdot g(n)$$



Figur 5: f skal altid ligge mellem de to funktioner med hver sin konstant.

Det betyder at funktion f skal altid ligge mellem de to funktion g , der har hver sin konstant, efter grænsen N_0 .

4.4.4 o (streng øvre grænse)

$$f(n) = o(g(n))$$

Hvis der for alle $c > 0$ findes en grænse N_0 , så der for alle n efterfølgende ($n \geq N_0$) gælder:

$$f(n) \leq c \cdot g(n)$$

Som også kan fortolkes som:

$$f(n) < c \cdot g(n)$$

Fungerer altså ligesom store o , men f må ikke længere vokse samme hastighed som g .

4.4.5 ω (streng nedre grænse)

$$f(n) = \omega(g(n))$$

Hvis der for alle $c > 0$ findes en grænse N_0 , så der for alle n efterfølgende ($n \geq N_0$) gælder:

$$f(n) \geq c \cdot g(n)$$

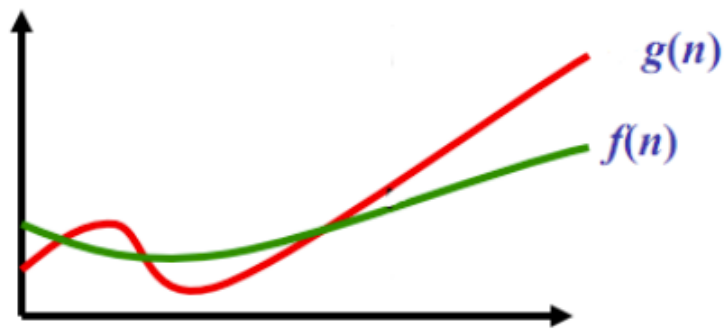
Som også kan fortolkes som:

$$f(n) > c \cdot g(n)$$

Fungerer altså ligesom store ω , men f må ikke længere vokse samme hastighed som g .

4.5 Sammenligning af voksehastigheder

For at sammenligne to algoritmers køretid skal vi sammenligne to funktioner, for at se hvordan hver algoritme vokser når inputtet n stiger.



Figur 6: Som ses på grafen overstiger $g(n)$ altid $f(n)$ på et tidspunkt, og derfor har $g(n)$ altså en større voksehastighed end $f(n)$.

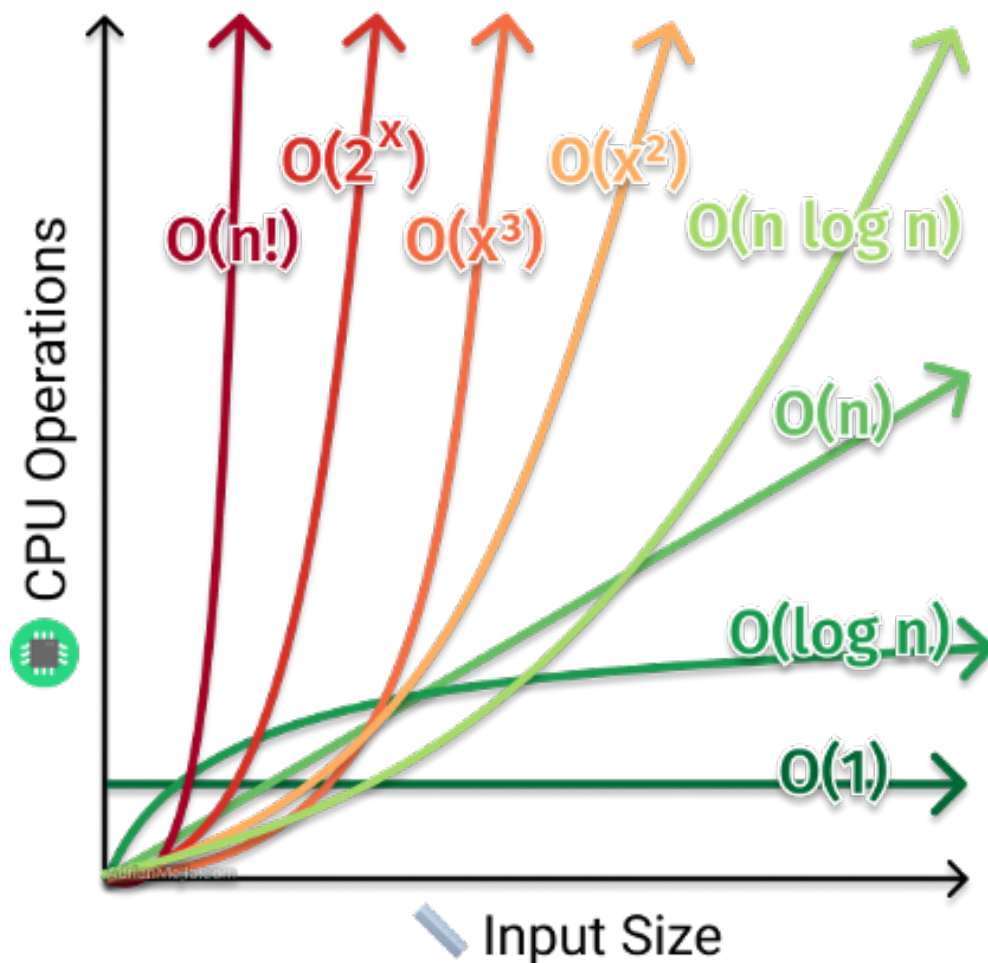
4.6 Vækststrategier

Voksehastigheder kan opstilles fra hurtigst til langsomst som følgende:

$$1, \quad \log n, \quad \sqrt{n}, \quad n, \quad n \log n, \\ n\sqrt{n}, \quad n^2, \quad n^3, \quad n^{10}, \quad 2^n$$

Figur 7: Voksehastigheder ordnet fra hurtigst til langsomst.

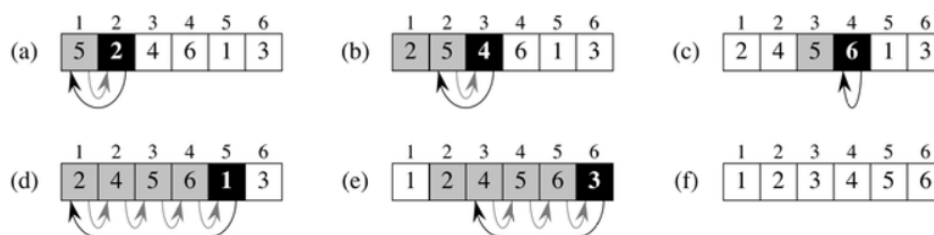
Visualisering af forskellige køretider:



Figur 8: Visualisering af forskellige køretider.

4.7 Invarianter

En invariant er et forhold som vedligeholdes af algoritmen i løbet af dens udførelse. Et eksempel på en invariant kan være i insertionsort, hvor alt til venstre for den nuværende key (sorte felt) skal være sorteret.



Figur 9: Invarianten i insertionsort: alt til venstre for den nuværende key er altid sorteret.

Oftentimes udgør invarianten selve kerneidéen af algoritmen, og kan også bruges som belæg for at algoritmen virker.

En invariant skal overholdes efter hvert skridt (iteration af løkke) udført af algoritmen. Dette kan bevises ved hjælp af induktion:



Figur 10: Induktionsprincippet: holder invarianten i basistilfældet, og bevarer hvert skridt den, så gælder den gennem hele kørslen.

4.8 Analyse af løkker i hånden

Noter tilføjes (udledning af Θ -køretid for indlejrede løkker).

4.9 Eksamenstips og faldgruber

Noter tilføjes.

5 Rekursionsligninger

5.1 Substitutionsmetoden

Noter tilføjes.

5.2 Rekursionstræ-metoden

Noter tilføjes.

5.3 Master-sætningen (CLRS 4. udgave)

Noter tilføjes (de tre tilfælde; hvornår den IKKE kan anvendes).

5.4 Eksamenstips og faldgruber

Noter tilføjes.

6 Grafgennemløb

6.1 Repræsentationer

Noter tilføjes (nabolister vs. nabomatrix).

6.2 Bredde-først søgning (BFS)

Noter tilføjes (d-værdier, besøgsrækkefølge).

6.3 Dybde-først søgning (DFS)

Noter tilføjes (discovery-/finish-tider; kantklassifikation: træ-/tilbage-/frem-/krydskanter).

6.4 Topologisk sortering

Noter tilføjes.

6.5 Eksamenstips og faldgruber

Noter tilføjes (konvention om alfabetisk ordnede nabolister).

7 Sortering

7.1 Sammenligningsbaserede sorteringer

Noter tilføjes (quicksort + PARTITION, merge sort, heapsort).

7.2 Lineærtids-sorteringer

Noter tilføjes (counting sort, radix sort, bucket sort).

7.3 Sammenligningstabel

Noter tilføjes (bedste/gennemsnitlige/værste tilfælde, in-place, stabil).

7.4 Eksamenstips og faldgruber

Noter tilføjes.

8 Minimum udspændende træ

8.1 Snit-egenskaben (cut property)

Noter tilføjes.

8.2 Kruskals algoritme

Noter tilføjes (hvilken kant kommer med næst; #sammenhængskomponenter undervejs).

8.3 Prims algoritme

Noter tilføjes.

8.4 Eksamenstips og faldgruber

Noter tilføjes.

9 Disjunkte mængder (Union-Find)

9.1 Operationer

Noter tilføjes (MAKE-SET, UNION, FIND-SET).

9.2 Union by rank

Noter tilføjes.

9.3 Path compression (stikomprimering)

Noter tilføjes.

9.4 Eksamenstips og faldgruber

Noter tilføjes (tegne skoven efter en sekvens af operationer).

10 Grådige algoritmer og Huffman-kodning

10.1 Det grådige princip

Noter tilføjes (grådigt valg, ombyttningsargument).

10.2 Huffman-kodning

Noter tilføjes (opbygning af træet, kodeordslængder, samlet antal bits).

10.3 Eksamenstips og faldgruber

Noter tilføjes.

11 Søgetræer

11.1 Binære søgetræer

Noter tilføjes (søgning, indsættelse, sletning, gennemløb).

11.2 Rød-sortede træer

Noter tilføjes (de fem egenskaber; tjek af gyldig farvning).

11.3 Indsættelse og rotationer

Noter tilføjes.

11.4 Udvidede (augmented) træer

Noter tilføjes.

11.5 Eksamenstips og faldgruber

Noter tilføjes.

12 Heaps og prioritetskøer

12.1 Heap-egenskaben og array-repræsentation

Noter tilføjes (forælder-/barn-indeks, min- vs. max-heap).

12.2 Opbygning af en heap

Noter tilføjes (BUILD-HEAP, HEAPIFY).

12.3 Operationer

Noter tilføjes (indsæt, extract-min/max, decrease-key).

12.4 Eksamenstips og faldgruber

Noter tilføjes.

13 Dynamisk programmering

13.1 Principper

Noter tilføjes (optimal delstruktur, overlappende delproblemer, memoisering vs. bottom-up).

13.2 Gennemregnede eksempler

Noter tilføjes.

13.3 Køretids- og pladsanalyse

Noter tilføjes (aflæse kompleksitet ud fra en rekursionsligning; reducere pladsforbrug).

13.4 Eksamenstips og faldgruber

Noter tilføjes.

14 Hashing og ordbøger (dictionaries)

14.1 Kædning (chaining)

Noter tilføjes.

14.2 Åben adressering

Noter tilføjes (linear probing, double hashing).

14.3 Fyldningsgrad (load factor) og ydeevne

Noter tilføjes.

14.4 Eksamenstips og faldgruber

Noter tilføjes (rekonstruere indsættelsesrækkefølge / mulige hashværdier).

15 Korteste veje

15.1 Relaksering (relaxation)

Noter tilføjes.

15.2 Dijkstras algoritme

Noter tilføjes (ingen negative kanter).

15.3 Bellman-Ford

Noter tilføjes (håndterer negative kanter; detekterer negative cykler).

15.4 Korteste veje i en DAG

Noter tilføjes.

15.5 Hvilken algoritme skal bruges

Noter tilføjes (valg ud fra negative kanter / cykler / DAG).

15.6 Eksamenstips og faldgruber

Noter tilføjes.